

EdgeCipline — Manual Test Plan

Use this as a checklist. Check items off as you go; anything that fails, note the steps + screenshot next to it so it's easy to hand back for a fix.

0. Before you start

Test accounts you'll need:

- ☐ A brand-new free-tier user (no trades logged yet) — for onboarding + free-limit flows
- ☐ A free-tier user with trades already logged in **both** Forex and Indian markets — to hit the 2-trade limit
- ☐ A premium/active-subscription user (either provider) — to confirm premium gates open
- ☐ Admin credentials (from `backend/scripts/ensureAdminUser.js` / whatever admin email you seeded)

Environment flags that change what you'll see (check `frontend/.env*` before you start so you're not chasing a "bug" that's actually a build flag):

- `NEXT_PUBLIC_PAYMENTS_ENABLED=true` → Razorpay checkout shows on **web**
- `NEXT_PUBLIC_PLAY_BILLING_ENABLED=true` → Google Play paywall shows on **native Android build only**
- If both are off, hitting the free-trade limit shows the plain "upgrade not available in this build" dialog, not a real checkout — that's expected, not a bug.
- Razorpay in non-production with no `NEXT_PUBLIC_RAZORPAY_KEY_ID`, or on native Android, uses a **sandbox mock checkout** (no real charge) — the modal says "Sandbox demo - no real charge".

Where to test each surface:

- Web flows (Forex/global market, Razorpay, admin panel): browser, desktop + mobile width.
- Indian market + Google Play Billing: only fully testable in the Capacitor Android build (emulator or device) with a Play Store test track. Web will show the Razorpay/mock path instead.

Part A — User side (Web)

A1. Auth & account lifecycle

- ☐ Register with a new email → verify OTP flow (`/register` → `/verify-otp`) works, resend OTP works, expired/wrong OTP is rejected with a clear message
- ☐ Login with correct credentials succeeds; wrong password is rejected without leaking whether the email exists
- ☐ "Forgot password" → reset email arrives → `/reset-password` link works → can log in with new password
- ☐ Accept-terms gate (`/accept-terms`) blocks app access until accepted, then never reappears

- ☐ Session persists across refresh and across new tab; logging out on one tab reflects in others (`AuthSessionBootstrap`)
- ☐ Onboarding flow (`/onboarding`) walks a brand-new user through creating a setup + logging their first trade (Step 2 of 3 banner shown on Add Trade)
- ☐ Delete account (`/delete-account`) actually removes access — try logging back in afterward, confirm it's blocked; confirm any account-deletion confirmation email/notice is correct

A2. Add Trade — Forex / global market (`/add-trade`)

Recently redesigned form — worth testing carefully field-by-field.

- ☐ Pair, Action (Buy/Sell), Lot size, Entry/Exit price, Net Profit all save correctly and appear on `/trades`
- ☐ Net Profit field turns green on positive / red on negative as you type
- ☐ Stop Loss / Take Profit save and show up on the trade detail/edit view
- ☐ **New setup field** ("Setup / Pattern", e.g. "Breakout above 1.0950") saves and displays separately from the `strategy` /Setup dropdown — confirm both fields persist independently on edit
- ☐ Planned Risk:Reward dropdown incl. "Custom" free-text (e.g. `1:2.5`) saves correctly
- ☐ Trade Date is bounded: can't pick before account-creation date or after today
- ☐ Commission / Swap fields save (Forex-only pair, replacing Indian's Brokerage/STT)
- ☐ Psychology block is enforced: mood, confidence, and emotional tags — try submitting without them and confirm validation blocks it with a clear message
- ☐ "Would you retake this trade?" toggle and Trade Quality (Great/Average/Poor) save
- ☐ Screenshot dropzone: upload an image, preview renders, spinner shows while uploading, saved trade shows the image
- ☐ Multi-image "Trade Evidence" section (separate from the single screenshot) — add multiple images, remove one, confirm final saved set matches
- ☐ Number fields reject invalid keystrokes (letters, `e` , multiple minus signs) via `blockInvalidNumberKeys`
- ☐ Submitting mid-upload (still uploading evidence) is blocked, not double-submitted

A3. Add Trade — Indian Market (`/indian-market/add-trade`)

- ☐ Instrument toggle "Options" vs "Intraday Stocks" switches the whole form correctly
- ☐ **Options:** Underlying dropdown incl. "Other" → free-text symbol; Strike price (integer); CE/PE; Buy/Sell; Qty (lots); Expiry date — all save
- ☐ **Equity:** Stock Symbol (uppercased), Exchange (NSE/BSE), Buy/Sell, Shares Qty, Sector (incl. Auto-detect) — all save
- ☐ Entry/Exit premium labels are correct per instrument type (e.g. "Avg buy/sell price" for equity vs "Entry/Exit premium (₹)" for options)
- ☐ **New Stop Loss / Take Profit fields** save and appear on trade view/edit (previously missing on this form)
- ☐ Trade type: Intraday / Delivery / Swing saves correctly (Swing is new — confirm it round-trips through edit and doesn't get coerced back to Intraday)

- ☐ Brokerage (₹) and STT/Taxes (₹) save
- ☐ **New single "Screenshot" dropzone** on this page (separate from multi-image Trade Evidence) — upload, preview, and confirm it lands on the trade record distinctly from Trade Evidence images
- ☐ Setup/Pattern free-text field saves independently of the Setup dropdown, same as Forex
- ☐ Same psychology-block validation (mood/confidence/emotional tags required) as Forex form
- ☐ Submit is blocked while screenshot is still uploading (`uploadingScreenshot` guard) — try clicking Save immediately after picking a screenshot

A4. Upload Trade / OCR screenshot extraction (`/upload-trade` , `/indian-market/upload-trade`)

A real bug was just fixed here (extraction returning the AI prompt's worked example instead of your actual trade) — test this thoroughly.

- ☐ Upload a clear, readable Forex broker screenshot (MT4/MT5 style) → extracted pair/entry/exit/profit/SL/TP match what's actually in the image, **not** a generic EURUSD/0.01 lot/+50 example
- ☐ Upload an Indian options screenshot → extracted strike/CE-PE/premium match the image
- ☐ Upload an Indian equity screenshot → extracted symbol/qty/prices match the image
- ☐ Upload a blurry/unreadable or non-trading screenshot → extraction **fails explicitly** ("could not extract any trade data") instead of inventing plausible-looking numbers
- ☐ Upload a screenshot with a currency pair/instrument the app hasn't seen before (not EURUSD) → confirm result isn't the stale sample trade
- ☐ After extraction, review/edit screen lets you correct any field before saving, and correction persists
- ☐ Batch upload of multiple screenshots at once (if supported) creates one trade per image, correctly attributed

A5. Free trade allowance & paywall (Web / Razorpay)

Free tier = **2 trades per market** (Forex and Indian Market tracked separately).

- ☐ As a fresh free user, log 2 Forex trades successfully; the 3rd attempt is blocked with a 402 and the `TradeLimitDialog` opens
- ☐ Dialog text is correct: "You've used your 2 free Forex trades" (or "Indian market" for that market) and remaining count is accurate
- ☐ Hitting the limit via **bulk import/upload** (asking for more entries than remain) shows the "That import needs N entries" variant and confirms **nothing** was partially saved
- ☐ Indian market and Forex limits are independent — using up Forex trades doesn't block Indian market trades and vice versa
- ☐ With `NEXT_PUBLIC_PAYMENTS_ENABLED=true` , hitting the limit opens the real SmartPaywall (Razorpay) instead of the plain dialog
- ☐ SmartPaywall shows correct, personalized metrics (discipline score, trades logged, best setup, etc.) pulled from the user's real data — a brand-new user with no data sees the "cold start" callout instead of empty/zero tiles

- ☐ Plan cards show current pricing tiers correctly (confirm against whatever the latest configured prices are — recently raised) with per-month breakdown and "SAVE X%" badges on longer plans
- ☐ Coupon code field: valid code discounts the payable amount and updates the pay button total live; invalid code shows "This code isn't valid" without crashing
- ☐ Complete a sandbox/test payment end-to-end → celebration screen (`PremiumWelcome`) shows correct plan label + expiry → paywall closes → app immediately reflects Premium (no stale "Free" state, no waiting for the 60s poll)
- ☐ Simulate a payment failure (`payment.failed` in Razorpay) → error shown, user is **not** charged twice, retry works
- ☐ Dismissing the paywall without paying (close button) doesn't break anything on reopen
- ☐ After the free allowance is hit, browsing/editing/viewing **existing** trades and analytics still works fully (only *creating new* trades is blocked)

A6. Promotions / attribution (user side)

- ☐ Visiting a referral/promo short link (`/r/[slug]`) redirects correctly and the attribution cookie/param is captured
- ☐ A valid promo/coupon applied at checkout reduces the price as configured by admin (percent and fixed-amount coupons both, if you have one of each configured)
- ☐ A disabled or expired coupon is rejected with the generic "not valid" message (no info leak about why)
- ☐ Reusing a single-use coupon a second time is rejected

A7. Dashboard & Analytics

- ☐ Dashboard loads with correct summary numbers (P&L, win rate, trades count) matching what you actually logged in A2/A3
- ☐ Switching market (Forex ↔ Indian) via the market switcher updates dashboard, trades list, and analytics scoped to that market only
- ☐ Analytics sub-pages load without errors and reflect real data: `analytics` , `discipline` , `mistakes` , `patterns` , `psychology` , `psychology-cost` , `self-awareness` , `trading-dna` , `ai-insights` , `ai-coach`
- ☐ Free user sees premium analytics sections gated/blurred with an upgrade prompt where applicable; premium user sees full data
- ☐ Weekly report (`/weekly-reports`) generates and reflects the correct week's trades
- ☐ Trading DNA share page (`/trading-dna/share`) produces a shareable view that doesn't leak more than intended

A8. Psychology / Discipline / Missions / Streaks / Checklist

- ☐ Discipline score changes sensibly after logging a rule-following vs rule-breaking trade
- ☐ Checklist (`/checklist` , `/checklist/psychology`) items persist and affect discipline scoring
- ☐ Missions (`/missions`) and Streaks (`/streaks`) update after relevant activity (e.g. daily login, trade logged)

- ☐ Coach chat (`/coach`) responds and quota pill correctly reflects remaining free coach queries for a free user

A9. Support (new ticketing system)

- ☐ `/support` shows contact channels and, when signed out, the correct sign-in-required state (`SignInRequired`)
- ☐ Create a new ticket (`/support/tickets/new`) with a message and an attachment → appears in `/support/tickets` list with correct status
- ☐ Open ticket detail (`/support/tickets/detail`) → send a follow-up message → message composer supports markdown rendering correctly
- ☐ Support widget (floating chat) opens/closes correctly and doesn't overlap other UI on mobile widths
- ☐ Browse a knowledge base article (`/support/article`) → article renders, and leaving feedback (helpful/not helpful) on it registers
- ☐ Ticket auto-close behavior (if idle) — check `supportAutoCloseCron` behavior isn't prematurely closing an active conversation (may need to wait or check with admin side)

Issue reports are support tickets (unified flow) — "Report Issue" anywhere in the app (OCR review screen, `/support` hub "Report a problem") opens a real support ticket:

- ☐ On the OCR review screen, tap "Report OCR Issue", add a description + 2 screenshots, submit → success screen shows a **ticket code (EC-XXXXXX)** and a "View ticket" button that lands on `/support/tickets/detail`
- ☐ That ticket appears in `/support/tickets` with a **"Bug report"** badge, category "Trade Import & OCR", and the opening message contains your description plus an "Extracted values: Symbol ... Entry ... Exit ..." footer
- ☐ The 2 screenshots are attached to the opening message and open via the ticket (they are private — pasting a screenshot's URL into a logged-out browser must 404)
- ☐ Reply on that ticket as the user → works exactly like a normal ticket (no difference from one created via "Open a ticket")
- ☐ Old `/issues` link → redirects to `/support/tickets` ; an old `/issues/detail?id=...` for a new report → redirects to its ticket
- ☐ Submit with **no network** → offline banner, submit disabled; reconnect → submit works and only **one** ticket is created (retry idempotency via `submissionId`)
- ☐ Double-tap Submit rapidly → only one ticket created
- ☐ With 10+ unresolved tickets already open (or lower `SUPPORT_MAX_OPEN_TICKETS_PER_USER` in env) → modal shows "You already have open tickets" with a "View my tickets" link, **no** ticket or issue row is created, and uploaded screenshots are not left orphaned in Cloudinary
- ☐ With `SUPPORT_TICKETS_ENABLED=false` → modal shows "Reporting is paused" with contact link and Submit is disabled
- ☐ Try attaching more than the support attachment cap (default 5) → capped client-side with a toast, and server rejects a 6th if forced
- ☐ Attach a non-image / HEIC file → rejected with a clear message, nothing half-submitted

- ☐ Report from the **Android app** (platform = android) → ticket's context shows "android vX.Y.Z" and Report from web shows "web"
- ☐ Report a non-OCR category (e.g. "Login") → ticket gets category "Account & Profile", priority **high**, and no "Extracted values" line (Crash → "Technical Issue", high; Journal/Setup → "Trading Journal", normal)

A10. Settings / Profile

- ☐ `/settings` (recently split out from `/profile`) shows and saves account settings correctly; `/profile` still shows profile info without duplicated/broken settings controls
- ☐ Notification settings (`/checklist/notification-settings` , `/notifications`) toggle correctly and actually suppress/enable the relevant notifications
- ☐ Changing password from settings works and requires current password
- ☐ Subscription status shown in settings matches reality (Free / Premium + correct expiry) immediately after any plan change from A5

A11. Mobile responsiveness (quick pass)

- ☐ Add Trade (both markets), paywalls, and support widget all usable at ~375px width — no cut-off buttons, no horizontal scroll
- ☐ Mobile bottom nav and user drawer (`MobileBottomNav` , `MobileUserDrawer`) open/close correctly and route to the right pages

Part B — Android app (Google Play Billing)

Only testable in the actual Capacitor Android build (internal test track or emulator with Play Store signed in to a license-tester account).

- ☐ Confirm **no Razorpay UI or code path is reachable** anywhere in the Android build (policy requirement — SmartPaywall must never render on native Android)
- ☐ Hitting the free-trade limit on Android opens `PlayBillingPaywall` , not SmartPaywall
- ☐ Plan list loads from Play (`edgeline-pro-monthly` / `-3month` / `-6month`) in the correct cheapest-first order, with Play's own localized prices — not any price from our backend
- ☐ Complete a real (license-tester / sandbox) purchase → verify step calls backend → entitlement flips to Premium → `PremiumWelcome` shows correct expiry
- ☐ Cancel out of the Play purchase sheet → paywall returns to "ready" state silently, no error shown
- ☐ Trigger a "pending" payment method (test card requiring delayed confirmation, if available in your test setup) → paywall shows the "Waiting for Google Play" panel, does **not** grant premium, and does **not** call it a failure
- ☐ "Restore purchases" with an existing active subscription on the device restores entitlement correctly
- ☐ "Restore purchases" with no purchase on the device shows "No previous Google Play subscription was found on this device"

- ☐ Sign in on a second device/account and attempt to restore a purchase that belongs to a different Edgecipline account → correctly shows the ownership-conflict message, not a generic error
 - ☐ Kill the app mid-purchase (after Play sheet opens, before verification completes), reopen → purchase is picked up and verified automatically on relaunch (reconcile-on-resume, `usePlayBillingReconcile`)
 - ☐ Google Play server-to-server notifications (renewal, cancellation, grace period, revoke) correctly update the subscription state in the DB — check via admin Payments/Users page after simulating each in the Play testing console
 - ☐ Double-tapping "Subscribe" does not open two Play purchase sheets
-

Part C — Admin side

Log in at `/admin/login` with seeded admin credentials.

C1. Admin auth

- ☐ Admin login succeeds with correct credentials, rejects wrong ones
- ☐ Non-admin user cannot reach any `/admin/*` route (try navigating directly while logged in as a regular user)
- ☐ Admin session expires/logs out correctly

C2. Admin dashboard (`/admin/dashboard`)

- ☐ Headline metrics (users, revenue, active subscriptions, trades logged) match what you'd expect from your test data
- ☐ Recently reworked dashboard — check for any stale/cached numbers by cross-checking one metric manually (e.g. count active premium users via Users page vs the dashboard tile)

C3. Users (`/admin/users`) & Expired users (`/admin/expired-users`)

- ☐ User list loads, search/filter works, pagination works
- ☐ Opening a user shows correct plan status, trade counts per market, and payment history
- ☐ Expired-users view correctly lists only users whose subscription has actually lapsed (test by expiring one via A5/B test payment and confirming it appears here after expiry)
- ☐ Any admin action here (e.g. manual plan grant/revoke, if present) takes effect immediately and is reflected on the user's own settings page

C4. Trades (`/admin/trades`)

- ☐ Trade list across all users loads, filters by market/user work
- ☐ Spot-check a trade you created in A2/A3/A4 appears here with correct fields, including the new `setup` , `stopLoss` / `takeProfit` fields

C5. Payments (/admin/payments)

- ☐ Every test payment from A5/B appears here with correct status, amount, and provider (Razorpay vs Google Play)
- ☐ Payment detail view shows enough to reconcile against Razorpay dashboard / Play Console (order/payment IDs, plan, amount)
- ☐ A pending/failed payment is distinguishable from a successful one at a glance

C6. Promotions (/admin/promotions) — new feature

- ☐ **Campaigns:** create a campaign (influencer / festival / general / new_user / other type), set draft → active → paused → ended, confirm status transitions are respected (e.g. a paused campaign's coupons stop working for new redemptions)
- ☐ **Coupons:** create a percent-discount coupon and a fixed-amount coupon, each tied to a campaign; confirm both work correctly at checkout (A6) and math matches (percent of the plan price vs flat rupee amount)
- ☐ Disable a coupon → confirm it immediately stops working at checkout
- ☐ **Influencers:** create one, active/inactive toggle, confirm redemptions attribute correctly to them
- ☐ Redemptions list shows every coupon use from your A6/A5 tests with correct user, coupon, and discount amount
- ☐ Overview/reporting numbers on this page match the redemptions list (no double-counting or missing entries)
- ☐ Attempt to create a coupon with invalid input (negative discount, >100% percent, duplicate code) is rejected with a clear validation error

C7. Support (/admin/support) — new feature

- ☐ Ticket list shows tickets created in A9, with correct status/priority
- ☐ Open a ticket (/admin/support/detail) → reply as agent → confirm the reply appears on the user's side (A9) in real time or on refresh
- ☐ Assign a ticket to an agent (listAgents) and confirm assignment reflects in the list
- ☐ Change ticket status (open → pending → resolved → closed) and confirm each transition is visible to the user and blocks/allows further replies appropriately
- ☐ Metrics endpoint/page shows sane ticket volume/response-time numbers
- ☐ **Articles** (/admin/support/articles): create/edit a knowledge base article, publish it, confirm it appears on the user-facing /support/article page; unpublish and confirm it's no longer visible to users
- ☐ Delete a ticket or article (if permitted) and confirm it's actually gone from both admin and user views
- ☐ Support audit log captures the actions above (if there's a visible audit trail)

Bug reports in the agent console (unified flow)

- ☐ The OCR ticket from A9 shows in the `/admin/support` queue with a **"Bug report"** badge and `bug / ocr_failure` tags, and staff got the normal "New ticket" notification (and **not** a second, separate issue alert)
- ☐ Open it → the context panel's top block is **"Bug report"** with issue code, category, screen/module, platform + app version, market, and an **OCR values table** (Extracted vs Corrected, changed rows highlighted); the issue code links to `/admin/issues/detail`
- ☐ As a plain **agent** (not admin) you can see that OCR table — it comes through the support context endpoint, not the admin-only issues API
- ☐ In `/admin/issues`, the same report shows a **Ticket EC-XXXXXX** chip linking to the ticket; legacy (pre-change) reports say "no support ticket"
- ☐ In `/admin/issues/detail`, set status **Investigating** → the ticket moves open → **In Progress** (visible to the user as "In Progress")
- ☐ Set status **Fixed** with a fix summary + version → the ticket gets a public "The bug behind this ticket has been fixed in vX..." reply from Support **and** becomes Resolved with that summary; the user receives the reply/resolved notifications only once (no duplicate "issue fixed" push)
- ☐ Set status **Closed** on the issue → the ticket is **not** touched (agents close tickets from the console, never yanked mid-conversation)
- ☐ Mark Fixed on an issue whose ticket the agent already **resolved/closed** → no error, no duplicate resolve, the issue still saves as Fixed
- ☐ Mark Fixed on a **legacy** report (no linked ticket) → the old "Issue Fixed" push still goes out and deep-links to a page that renders

C8. Feedback / Issues / Monitoring

- ☐ `/admin/feedback` shows feedback submitted by users (including article feedback from A9)
- ☐ `/admin/issues` and `/admin/issues/detail` show issue reports raised by users (`/issues`, `/issues/detail` on the user side) with correct detail and status updates syncing both ways
- ☐ `/admin/monitoring` reflects real system health/activity (not stale placeholder data)

Part D — Production Readiness / Go-Live Verification

This is separate from feature QA above: everything in Part A–C can pass and the app can still be unsafe or broken to actually launch. Go through this before flipping real traffic onto it.

D1. Environment & configuration

- ☐ Production `.env` (backend) has **no test/sandbox secrets**: real Razorpay live keys (not `rzp_test_*`), real Google Play service-account credentials, real Mongo URI — not a local/staging DB

- ☐ `NEXT_PUBLIC_PAYMENTS_ENABLED` / `NEXT_PUBLIC_PLAY_BILLING_ENABLED` are set correctly **per build target** (web build vs Android build) — a wrong flag here is a store-policy violation, not just a UI bug (see `frontend/config/payments.js`)
- ☐ `NEXT_PUBLIC_RAZORPAY_KEY_ID` in the production web build is the **live** key, and the sandbox mock checkout path (`utils/mockRazorpay`) is confirmed **unreachable** in production (`NODE_ENV=production` short-circuits it — verify the built bundle doesn't expose it)
- ☐ CORS allowlist (`backend/config/index.js`) contains only the real production frontend origin(s) — no `localhost` , no old staging/Cloudflare Workers preview URLs left in from testing
- ☐ `CORS_DEBUG` is off in production
- ☐ All external URLs (API base URL, Cloudinary, mail service) point at production resources, not staging/test ones
- ☐ JWT/session secrets, admin credentials, and any API keys are unique to production (not reused from staging/dev) and not committed to the repo
- ☐ Database has proper indexes applied (`backend/scripts/syncIndexes.js`) before go-live — check trade queries and support/promotion collections aren't doing full scans under load

D2. Security

- ☐ Every `/admin/*` API route rejects a non-admin JWT with 401/403 — spot-check a few directly (not just the frontend route guard, which is bypassable)
- ☐ Razorpay webhook (`razorpayWebhookRoutes.js`) verifies the signature and rejects a forged/unsigned payload
- ☐ Google Play real-time developer notifications (`googlePlayNotificationRoutes.js`) verify the Pub/Sub token/signature and reject spoofed notifications
- ☐ IDOR check: as User A, try to fetch/edit/delete User B's trade, ticket, or payment by guessing/incrementing an ID — confirm every one is rejected
- ☐ Password reset and OTP tokens are single-use and expire; a used/expired token is rejected
- ☐ Rate limiters (`backend/middleware/rateLimiter.js`) are active in production config for login, OTP, payment, and support-polling endpoints — not accidentally left in a permissive dev mode
- ☐ File upload (trade screenshots, support attachments) rejects non-image/oversized files and doesn't allow path traversal or executable uploads
- ☐ All cookies (session/auth) are `Secure` + `HttpOnly` + `SameSite` appropriately set when served over HTTPS
- ☐ No stack traces, internal error messages, or secrets leak in any API error response in production mode
- ☐ Confirm HTTPS is enforced end-to-end (no mixed content, no plain-HTTP fallback for API calls)

D3. Payments — compliance & reconciliation

- ☐ A real ₹1 (or smallest real plan) live Razorpay transaction completes end-to-end and shows correctly in both the Razorpay dashboard and `/admin/payments`

- ☐ Google Play production listing's subscription products/base plans match exactly what `backend/constants/googlePlay.js` expects (product ID, base plan IDs, order) — a mismatch here silently breaks purchases for every Android user
- ☐ Refund/cancel flow (via Razorpay dashboard or Play Console) correctly downgrades the user's entitlement within a reasonable time, without manual intervention
- ☐ Confirm the app is genuinely free of any third-party payment processor inside the Android build — re-verify B's "no Razorpay reachable on Android" check specifically against the production Android build artifact, not just dev
- ☐ Pricing shown to users (₹349 / ₹899 / ₹1499 tiers, or whatever is currently configured) matches what's actually charged — cross-check `getPaywallContext` /plan config against the real Razorpay plan/order amounts and the Play Console base plan prices
- ☐ Webhook reconciliation cron (`webhookReconciliationCron.js`) runs on schedule in production and actually catches a manually-simulated missed webhook

D4. Data integrity & backups

- ☐ Automated database backups are configured and a restore has actually been tested at least once (not just "backups are running")
- ☐ The new `Trade.setup` field and other recent schema additions don't break reads of **pre-existing** trades created before this change (they should just show blank/default, not error)
- ☐ Account deletion actually removes/anonymizes personal data per your privacy policy (`/privacy-policy`) — check no orphaned personal data remains in trades, payments, or support tickets after a deletion in production-like data
- ☐ Migration/seed scripts (`ensureAdminUser.js` , `seedKnowledgeBase.js`) have been run against production and produce the expected admin user + articles, without duplicating existing data if run twice

D5. Performance & reliability

- ☐ Core pages (dashboard, trades list, add-trade) load in an acceptable time against production data volumes, not just an empty test DB
- ☐ Trade list / analytics pages remain responsive for a user with hundreds of trades (test with a seeded heavy account, not just 2-3 trades)
- ☐ OCR/AI extraction has a sane timeout and shows a clear error rather than hanging if the AI provider is slow/down
- ☐ Basic load check on login/payment endpoints (a handful of concurrent requests) doesn't error or corrupt state (see `paymentConcurrency.test.js` for what's already covered by automated tests — confirm it holds under real network conditions too)
- ☐ App behaves reasonably when the database or AI provider is briefly unreachable — user sees an error, not a blank white screen or infinite spinner

D6. Observability

- ☐ Error tracking/logging (`backend/utils/logger.js` output, or whatever APM is wired up) actually captures a deliberately-triggered error in production
- ☐ `/admin/monitoring` reflects a real, deliberately-caused event (e.g. a failed payment or a 500) within an acceptable delay
- ☐ Uptime/alerting is configured for the API and frontend, and you've confirmed at least one test alert actually reaches someone
- ☐ Logs don't contain plaintext passwords, tokens, or full payment card/payment-provider secrets

D7. App store / platform compliance

- ☐ Android build submitted for review has Play Billing as the **only** purchase path, matches the in-app declared subscription products, and privacy policy / data-safety form match actual data collected
- ☐ Play Store listing screenshots/description match the actual current app (not stale pre-redesign screenshots, given the Add Trade form change)
- ☐ If iOS is planned, confirm Razorpay/payments are fully compiled out per the comment in `frontend/config/payments.js` (both false, until StoreKit is implemented)

D8. Rollback & incident readiness

- ☐ You know exactly how to roll back the current deploy (previous build/tag identified) if something breaks post-launch
- ☐ A runbook or at least a clear owner exists for "payments are failing in production" and "OCR/AI provider is down" scenarios
- ☐ Support team (or you) has access to `/admin/support` and `/admin/payments` in production before go-live, not set up after the fact

D9. Final go/no-go

- ☐ All Part A/B/C items are checked, or any unchecked ones are explicitly accepted as known issues with an owner and a fix date
- ☐ All D1–D8 items above are checked — these are the ones that turn a "works on my machine" app into a production incident
- ☐ Someone other than the person who built the feature has run through Part A once, end to end, without help

Cross-cutting things worth breaking on purpose

- ☐ Rate limiting: hammer the OTP, login, or payment endpoints quickly and confirm you get throttled with a sane error, not a crash
- ☐ Refresh mid-flow (mid-payment, mid-OCR-upload, mid-ticket-reply) — confirm nothing double-submits and nothing gets stuck in a permanent loading state

- ☐ Log out and back in immediately after a plan upgrade — Premium status must survive re-login
- ☐ Try submitting the Add Trade / Upload Trade forms with the network throttled ("Slow 3G" in devtools) — loading/disabled states should hold, not let you double-submit
- ☐ Try editing a trade's disallowed/read-only fields directly via API (if you're comfortable with devtools/Postman) to confirm the field allowlist (`tradeFieldAllowlist.js`) actually rejects fields it shouldn't accept, including on both `setup` (new) and anything not in the list

Sign-off

Area	Tested by	Date	Result
Auth & onboarding			
Add Trade (Forex)			
Add Trade (Indian)			
OCR upload			
Free limit + Razorpay paywall			
Google Play Billing			
Analytics/Psychology			
Support (user)			
Admin — Users/Trades/Payments			
Admin — Promotions			
Admin — Support			
Production readiness (Part D)			